

AI-Powered Unified Mobility and Delivery System Using Dynamic Feasibility Analysis

Sivasubramanian M

*Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
sivamah25@gmail.com*

S. R. Ramya

*Assistant Professor/CSE
Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
srramyacse@gmail.com*

Rakshana S

*Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
rakshanasanjith05@gmail.com*

Sadhana T

*Dept. of Computer Science and Engineering
Dr. N.G.P. Institute of Technology
Coimbatore, Tamil Nadu, India
sadhanathangamuthu16@gmail.com*

Abstract—Ride-hailing, food delivery and parcel courier platforms move vehicles over the same street network but plan in isolation, so three vehicles are often dispatched where one coordinated trip would do. This paper presents a unified pipeline built around a Dynamic Multi-Service Feasibility Engine (DMFE), which decides whether a passenger ride, a food order and a parcel drop can share a vehicle before any driver is committed or any route is planned. Candidate pairs are scored on pickup proximity, route similarity, time-window overlap, vehicle capacity and request priority; admitted batches are assigned by a five-factor driver selector and then routed by a constraint solver. An adaptive mode recomputes the scoring weights each dispatch cycle from operating context and from bounded corrections derived from completed-trip residuals. Evaluation used reproducible synthetic workloads of 50, 100, 250 and 500 requests over 36 seed-matched runs with a fixed 60-vehicle fleet, against an independent-dispatch baseline. Where both configurations served the same requests, batching cut dispatched trips by 36.8% and 40.4% and raised mean vehicle utilisation by 17.0 and 17.4 percentage points, at mean service delays of 5.8 and 6.1 minutes against a 20-minute cap. Consolidation within the assigned fleet lowered estimated CO₂ by 22.0% to 43.9% against serving the same requests unbatched. The adaptive mode improved consolidation at every workload and raised mean pair compatibility from 55.4 to 59.2. The evaluation is synthetic and single-region. The adaptive layer utilizes a bounded residual correction, which was empirically validated to improve batch compatibility. The gradient-based learning formulation is a proposed extension.

Index Terms—Adaptive systems, dynamic feasibility analysis, food delivery, parcel delivery, ride sharing, route optimisation, unified mobility, vehicle routing.

I. INTRODUCTION

A. Background and Motivation

Three parallel economies now run over one street network. A passenger opens a ride-hailing app, a diner opens a food-delivery app, a shopper books a courier, and each of those requests is matched, routed and closed out inside a platform

that cannot see the other two. At peak hour the predictable consequence is three vehicles from three operators working the same two blocks minutes apart, each carrying one job.

The waste is not a failure of any individual matching algorithm. It follows from treating passenger mobility, food delivery and parcel logistics as separate problems, when to a road network they are one flow of vehicle movements under three commercial labels. Shareability analysis of city-scale taxi records showed that a large share of trips can be merged under modest delay tolerances [1], and dynamic vehicle-routing research showed that re-optimising as requests arrive outperforms static planning [7]. Neither addresses a pool of three different kinds of job at once, each with its own pickup logic, timing sensitivity and capacity requirement.

The missing piece is a decision layer that settles, ahead of any routing, whether two pending requests could realistically travel together. Geography alone will not settle it. Two requests raised a block apart are still a poor pairing when one is a meal with a short delivery window and the other a long cross-town ride. This paper builds that layer and measures what it does under controlled, reproducible load.

B. Contributions

- A Dynamic Multi-Service Feasibility Engine (DMFE) that scores every candidate pair of pending requests on pickup proximity, route similarity, time-window overlap, vehicle capacity and request priority before any route is computed, embedded in a unified pipeline spanning passenger transport, food delivery and parcel delivery instead of three independent services.
- An adaptive weight-generation mode, A-DMFE, in which the five scoring weights are recomputed each dispatch cycle from measurable operating context and from bounded corrections refitted out of completed-trip residuals, so the balance between criteria is not fixed by hand.
- A reproducible evaluation over 36 seed-matched runs at four workload sizes against an independent-dispatch baseline on identical request pools, reporting first-wave

dispatch, batching, utilisation, delay and consolidation separately rather than as a single headline figure.

- An explicit account of what the evaluation does not establish: which comparisons are like-for-like, where the fleet rather than the algorithm bounds throughput, and which parts of the mathematical formulation are proposed rather than trained.

II. RELATED WORK

A. Joint Passenger and Goods Transport

Surveys of ridesharing and crowdsourcing for smart cities map the layered architecture such systems assume, from communication infrastructure through matching and routing [2]. They give a structural account rather than an algorithmic one: feasibility appears as a concept in the layering, seldom as a step with a formulation of its own.

The closest precedent is the Autonomous Vehicle Intelligent System, which formulates joint ride-sharing and parcel delivery as a mixed-integer linear program deciding which passenger and parcel requests an autonomous vehicle should serve together [3], scoped to autonomous fleets and two service types. The work here targets a human-driver fleet and admits a third category, food delivery, whose much tighter windows change what counts as a feasible pairing. Reinforcement-learning approaches handle joint matching, pricing and dispatching [4] and joint passenger-and-goods fleet management [5] end to end, which is powerful but exposes no inspectable feasibility criterion; crowdsourced multi-hop parcel networks show a parallel demand for flexible fulfilment [6].

B. Dynamic Routing and Request Matching

A recurring finding in dynamic vehicle-routing research is that allowing the request set to grow during execution produces measurably better routes than freezing it when planning starts [7]. That stance is inherited here: compatibility is recomputed as arrivals land, and a request stays eligible to join a batch until it is dispatched. Iterated-matching work on food-delivery rider allocation isolates how narrow the timing margin for meal orders is [8]: a pairing workable for a passenger collapses once one leg must still be hot on arrival. Collaborative frameworks combining distinct vehicle modes report that hybrid fulfilment can beat single-mode fleets [9], and hybrid meal-delivery models built on adaptive large-neighbourhood search handle the dynamic mixed-vehicle setting on-demand delivery actually operates in [10]. Both support using a constraint-programming solver [11] for routing, since it accommodates the mixed capacity, time-window and priority constraints such a fleet imposes.

C. Research Gap

Table I positions the proposed system against these works along four dimensions: coverage of more than one request category, placement of the admissibility test ahead of routing, tolerance of arrivals appearing after planning has begun, and whether the system can state why a pair was accepted or rejected. No reviewed system combines all three service types

under a feasibility-first, dynamically-updating compatibility layer that also emits a per-pair factor attribution. That combination is the gap addressed here.

TABLE I
POSITIONING OF THE PROPOSED SYSTEM AGAINST RELATED WORK

System	Multi-service	Feasib. first	Dynamic arrival	Per-pair reasons
Smart-city ridesharing surveys [2]	Conceptual	No	Varies	No
Joint ride/parcel MILP (AVIS) [3]	Two types	Yes	No	No
RL fleet management [4], [5]	Two types	Implicit	Yes	No
Dynamic VRP with pickup [7]	No	No	Yes	No
Food-delivery matching [8]	No	Partial	Yes	No
Hybrid multi-mode routing [9], [10]	Two modes	No	Yes	No
Proposed DMFE	Three types	Yes	Yes	Yes

Entries summarise the scope reported by each work. They are the authors' reading of those papers, not a re-implementation or re-measurement of them.

III. PROPOSED METHODOLOGY

A. System Architecture

Fig. 1 shows the pipeline. Requests from all three categories enter a normalisation module that maps each one onto a common schema recording origin, destination, requested time, payload type, capacity demand and priority, so every downstream stage reasons over one representation rather than three. Normalised requests accumulate in a pending queue that each dispatch pass drains in arrival order. A cheap pre-filter drops pairs failing on pickup radius, time window or capacity before any scoring is done; survivors are scored on five factors and passed through the admission gates. An admitted batch then receives a driver and vehicle from a five-factor selector, and only afterwards is its route optimised by the OR-Tools constraint-programming solver [11] against that vehicle's capacity and fuel efficiency. Distances come from the Google Maps Distance Matrix API when a key is configured and from a local haversine matrix otherwise. Completed trips are logged with their factor attribution, and in adaptive mode those outcomes return to the context and learning modules.

The stage ordering is deliberate. Computing feasibility after routes are fixed can only accept or reject the batches routing happened to construct, rather than searching the pending pool for the best available pairing, and running a full solve over every candidate pair wastes work the pre-filter removes for the cost of three comparisons. A request that clears no partner is served on the single-service path rather than forced into a poor match.

B. Compatibility Scoring

For a candidate pair of pending requests (i, j) the engine forms a compatibility score as a weighted combination of five factor scores, each in [0, 1], expressed as a percentage of the attainable maximum:

$$CS(i, j) = 100 \times \frac{\sum_f w_f F_f(i, j)}{\sum_f w_f} \quad (1)$$

with f ranging over the five factors defined below and $w_P = 0.30$, $w_R = 0.25$, $w_T = 0.20$, $w_V = 0.15$ and $w_Q = 0.10$ in the deployed configuration. The pair is admitted for batching when

$$\text{admit}(i, j) \iff CS(i, j) \geq \tau \wedge V(i, j) > 0 \quad (2)$$

where $\tau = 70$. The capacity factor doubles as a hard gate: a pair whose combined demand exceeds the vehicle envelope scores zero and is excluded regardless of its other four factors.

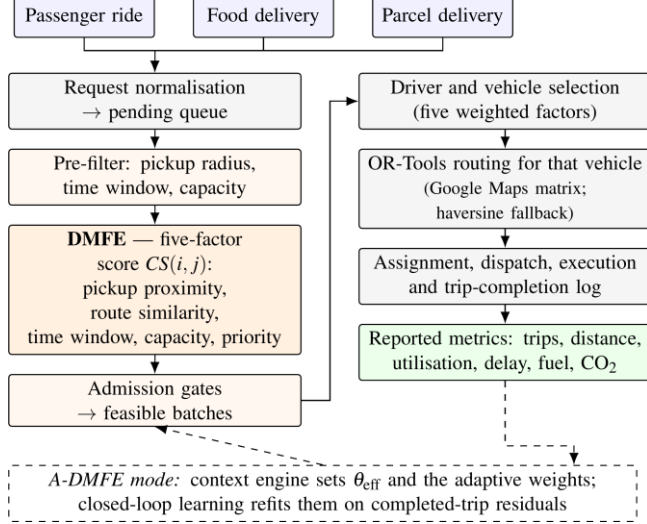


Fig. 1. Implemented DMFE dispatch pipeline. The driver and vehicle are selected before the route is optimised. Dashed arrows carry the context and learning feedback active only in A-DMFE mode.

Pickup proximity decays linearly with the great-circle distance d_p between the two pickup points, reaching zero at the search radius $R_{\max} = 5$ km:

$$P(i, j) = \max \left\{ 0, 1 - \frac{d_p(i, j)}{R_{\max}} \right\} \quad (3)$$

Route similarity averages a direction term and an overlap term. The direction term maps the cosine of the angle θ between the two origin-to-destination vectors onto $[0, 1]$. The overlap term averages three gap proxies, taken between pickups, between trip midpoints and between drop-offs, each normalised by the mean trip length:

$$R(i, j) = \frac{1}{2} (S_{\text{dir}} + S_{\text{ovl}}), \quad S_{\text{dir}} = \frac{1}{2} (1 + \cos \theta_{ij})$$

$$S_{\text{ovl}} = \frac{1}{3} \sum_{g \in \{p, m, d\}} \max \left\{ 0, 1 - \frac{\delta_g}{\ell} \right\} \quad (4)$$

Time-window overlap decays linearly in the gap between the two requested service times and reaches zero at the maximum allowed delay W :

$$T(i, j) = \max \left\{ 0, 1 - \frac{|t_i - t_j|}{W} \right\} \quad (5)$$

In the present implementation W is a single global constant of 20 minutes applied to all three service types. Service-specific tolerance windows are part of the generalised formulation — replacing W by a per-type W_{type} makes the score $1/W_{\text{type}}$ -Lipschitz in the requested time, so a tighter window penalises the same timing offset more heavily — but per-type values are not separately configured in the system evaluated here, and no per-type result is claimed.

In practice a per-type window would be derived from the urgency of each service and from the delay its customers will accept. Food orders warrant the tightest window, because a late arrival degrades the delivered product itself; passenger trips tolerate more slack, bounded by an acceptable pickup delay; parcels can usually wait longest, subject to the operator's own service policy. Such values should be taken from service-level requirements and confirmed by sensitivity testing rather than chosen arbitrarily. For the evaluation reported here all three services share the single 20-minute cap, which keeps the experimental conditions identical across service types.

Capacity is graded in four steps on the tighter of the seat and weight ratios, and is the factor that makes the score fail closed:

$$V(i, j) = \begin{cases} 1.0, & u_{ij} \leq 0.5 \\ 0.7, & 0.5 < u_{ij} \leq 0.8 \\ 0.3, & 0.8 < u_{ij} \leq 1.0 \\ 0.0, & u_{ij} > 1.0 \end{cases}$$

$$u_{ij} = \max \left\{ \frac{c_i + c_j}{C_{\max}}, \frac{m_i + m_j}{M_{\max}} \right\} \quad (6)$$

where c and m are the seat and weight demands of the two requests and $C_{\max}$, $M_{\max}$ the batch envelope of 6 seats and 100 kg. Priority is the mean of the two requests' priority values on a three-level scale:

$$Q(i, j) = \frac{1}{2} (\rho_i + \rho_j), \quad \rho \in \{0.2, 0.6, 1.0\} \quad (7)$$

so that batching throughput is not bought by routinely deferring urgent work. Because every factor lies in $[0, 1]$ and the weights are non-negative and normalised, the score is a convex combination, and τ therefore carries the same meaning for any two pairs however unlike.

C. Admission, Driver Selection and Route Optimisation

A batch is admitted when the pair clears the score threshold and both the capacity and time factors remain strictly positive, and when no high-priority request in the batch would exceed its delay budget. Admitted batches and unbatched requests then follow the same two-stage dispatch. The order matters and is the reverse of what a routing-first design would do: the driver and vehicle are chosen first, and the route is optimised afterwards for that specific vehicle, because the routing objective prices fuel through the vehicle's own efficiency and the capacity constraint depends on the vehicle actually assigned. For a batch B the selector takes

$$(k, v)^* = \arg \max_{k \in \mathcal{D}} \sum_{a=1}^5 \gamma_a \Phi_a(k, B) \quad (8)$$

over the available drivers, where the five factor scores Φ_a cover the driver's ETA to the first pickup, the match between vehicle type and batch composition, current workload, rotation fairness and historical completion rate, with coefficients $\gamma = (0.50, 0.30, 0.20, 0.10, 0.15)$. These coefficients form a raw weighted sum rather than a convex combination, so the selection score is ordinal within a dispatch cycle. In adaptive mode the proximity coefficient carries a bounded bump when the learning component reports a corridor running late. The rule is greedy per batch; the globally optimal alternative is the classical linear assignment problem, solvable exactly by the Hungarian method [12], which the implementation does not use because the per-cycle latency budget favours the greedy rule.

With the vehicle fixed, the routing stage searches for the pickup and drop-off ordering π that minimises a combined distance, time and fuel cost with a discount for high-priority pickups:

$$\begin{aligned} \pi^* &= \arg \min_{\pi \in \Pi(B)} c(\pi) \\ c(\pi) &= \sum_{(u,v) \in \pi} c_{uv} - b n_{hi}(B) \\ c_{uv} &= d_{uv} + w_t t_{uv} + w_f p_f d_{uv} / \eta \end{aligned} \quad (9)$$

Here d_{uv} is the arc distance in metres, t_{uv} the arc duration in seconds, η the assigned vehicle's rated efficiency, p_f the fuel price, n_{hi} the number of high-priority pickups, and b the meter-equivalent discount applied once at each of them. The deployed coefficients are $w_t = 0.3$, $w_f = 1.0$ and $b = 2000$ m; an optional per-kilometre operating-cost term exists but is disabled. The solver receives the vehicle capacity and a duration cap equal to the 20-minute delay budget, with a two-minute service time at each pickup. Algorithm 1 states one dispatch pass end to end, marking in the right margin the equation each step realises.

Algorithm 1 DMFE single dispatch pass

Input: pending pool of requests, driver pool D , configuration
1: $w \leftarrow \text{GENERATEWEIGHTS}(\text{mode}, \text{context}, \text{learned bias})$ — (10)
2: $E \leftarrow \emptyset$
3: for each unordered pair (i, j) drawn from the pool do
4: if $V(i, j) = 0$ then continue — (6)
5: evaluate P, R, T, Q ; $CS(i, j) \leftarrow \sum_f w_f F_f(i, j)$ — (1), (3)–(7)
6: if $CS(i, j) \geq \tau$ then $E \leftarrow E \cup \{(i, j, CS)\}$ — (2)
7: $B \leftarrow \text{GREEDYBATCH}(E)$ taking edges by descending CS , capacity-feasible
8: for each batch $b \in B$ and each unbatched request do
9: $k, v \leftarrow \text{SELECTDRIVER}(b, D)$ — (8)
10: $\pi \leftarrow \text{ORTOOLSROUTE}(b, v)$ — (9)
11: dispatch(b, π, k, v); log per-factor attribution
12: on trip completion: record residual; refit every 200 trips — (11)

D. Adaptive Weight Generation and Closed-Loop Learning

Treating the five weights as hand-tuned constants is what separates a rule-based filter from a system that improves with operation. In adaptive mode the weights are regenerated once

per dispatch cycle from a base vector, a context perturbation and a learned correction, then renormalised:

$$\begin{aligned} \hat{w}_f &= w_f^0 (1 + \Delta_f^{\text{ctx}}) (1 + \Delta_f^{\text{lrn}}), \quad |\Delta_f^{\text{lrn}}| \leq B \\ w_f &= \hat{w}_f / \sum_g \hat{w}_g \end{aligned} \quad (10)$$

The context term shifts weight with measurable conditions: traffic and fuel pressure raise the route weight, a deep pending queue raises the pickup weight, driver scarcity raises the capacity weight, peak demand raises the time weight, and a concentration of urgent requests raises the priority weight. Each response is a calibrated linear ramp on a bounded signal, so the mapping from condition to weight is deterministic and can be reported alongside any decision.

The learned term is produced by a closed-loop component that records, for every completed trip, the residual between the delay and utilisation the engine predicted and what the trip actually returned. Residuals are bucketed by service corridor — the sorted mix of request types in the batch, such as food and ride — and every 200 completed trips a per-corridor correction factor is refitted as a damped step toward the ratio of realised to predicted values:

$$\begin{aligned} \mu_c^{(n+1)} &= \text{clip}[\mu_c^{(n)} + \lambda(r_c - \mu_c^{(n)}), \mu_{\min}, \mu_{\max}] \\ r_c &= \sum_{j \in \mathcal{B}_c} y_j / \sum_{j \in \mathcal{B}_c} \hat{y}_j \end{aligned} \quad (11)$$

with damping $\lambda = 0.5$ and clip bounds $[0.5, 2.0]$. The refitted factors feed the delay and utilisation predictions, and the sign of the accumulated residual drives the learned weight correction, bounded at $B = 0.15$ so no single factor can be driven far from its configured value. This mechanism is a bounded heuristic correction computed on measured residuals. It is not gradient descent, and it is not described as such anywhere in this paper.

E. Proposed Learning Extension

A natural next step, formulated here but neither implemented nor trained, replaces the bounded correction with a directly optimised weight vector. Generating the weights from an unconstrained latent vector θ by softmax keeps them on the probability simplex automatically, and treating the retrospective verdict on a completed batch as a binary label admits a cross-entropy objective over the normalised score:

$$\begin{aligned} \alpha_f &= \exp(\theta_f) / \sum_g \exp(\theta_g) \\ \mathcal{L}(\theta) &= -[\ell \log \widetilde{CS} + (1 - \ell) \log(1 - \widetilde{CS})] \\ \theta_f &\leftarrow \theta_f - \eta \partial \mathcal{L} / \partial \theta_f \end{aligned} \quad (12)$$

The relations in (12) describe a proposed extension. No softmax parameterisation, cross-entropy objective or gradient update was executed in any experiment reported in this paper, and no learned weight vector is presented as a result.

F. Complexity and Cost

Scoring is pairwise, so an unrestricted pass over a pool of size n costs $O(n^2)$ factor evaluations. The measured pair counts

follow that shape — 489, 1,817, 11,049 and 44,095 per run at the four workloads, an empirical growth exponent of 1.96. A bounded neighbourhood of size k would reduce this to $O(nk)$; the 5 km pickup radius provides such a bound in principle, but at these densities it did not bind. Per-request processing nonetheless fell from 15.4 ms at 50 requests to 8.3 ms at 250 in static mode, the fixed per-cycle overhead amortising over more requests.

IV. EXPERIMENTAL SETUP

A. Workload Generation

Each run generates a fresh pool of synthetic requests over a bounded urban area using the platform's own request generator, with a mix of 40% passenger rides, 40% food orders and 20% parcel drops. Every request carries an origin, a destination, a requested time, a capacity demand and a priority level. Workloads of 50, 100, 250 and 500 requests were run with five seeds each at the three smaller sizes and three at 500, giving 36 runs across the two engine modes. Seeds are shared between the static and adaptive arms and between the engine and the baseline, so every comparison in Section V is paired on identical request pools.

This is a controlled synthetic workload, not a field deployment. It is designed to answer how the engine behaves as load grows under conditions that can be regenerated exactly, and it does not support claims about real demand patterns, driver behaviour or traffic.

B. Fleet and Engine Configuration

Table II lists the configuration. The fleet is fixed at 60 vehicles across five types with the fuel efficiencies shown, and is held constant across all four workloads; that choice is what produces the throughput ceiling analysed in Section V-C. The distance layer supports the Google Maps Distance Matrix API, but the evaluation ran with the API key unset so that the deterministic haversine fallback is used. All 1,869 trips across the 36 runs were computed on haversine distances scaled by a road factor of 1.25, and no external request was issued. The experiments are therefore exactly reproducible, and correspondingly they do not capture live traffic.

TABLE II
EVALUATION CONFIGURATION, READ FROM THE ENGINE CONFIGURATION RECORD

Parameter	Value
Scoring weights w_P, w_R, w_T, w_V, w_Q	0.30, 0.25, 0.20, 0.15, 0.10
Admission threshold τ	70 of 100
Maximum pickup radius $R_{\max}$	5 km
Global maximum allowed delay W	20 min
Batch envelope $C_{\max}, M_{\max}$	6 seats, 100 kg
Adaptive learned-bias bound B	0.15
Fleet (fixed at all workloads)	60 vehicles at 40 / 25 / 15.5 / 13 / 8 km/L: 24 bikes, 10 autos, 16 cars, 8 vans, 2 trucks
Request mix	40% ride, 40% food, 20% parcel
Workloads $\times$ repetitions	50, 100, 250 (5 seeds); 500 (3 seeds)
Road factor, mean speed, service time	1.25, 25 km/h, 2 min per stop
Emission factor	2.3 kg CO ₂ per litre
Driver / routing coefficients	$\gamma = 0.50, 0.30, 0.20, 0.10, 0.15$; $w_t = 0.3, w_f = 1.0, b = 2000$ m
Route solver / distance source	Google OR-Tools / haversine matrix

Values are read from the configuration recorded inside each run. The matrix ran in haversine mode for all 36 runs so the experiments reproduce offline; the Google Maps integration exists in the system but was not exercised here.

C. Baseline

The primary baseline is independent single-service dispatch: every request is served on its own, with no cross-service batching. Its distance is the road-factored great-circle distance from origin to destination, and its vehicle is the type that service would ordinarily assign to that request category. To evaluate algorithmic scalability, we also compared against the platform's legacy monolithic joint-optimization solver (AIOrchestrator), which solves the entire batch as a single VRP. While the monolithic solver successfully served $N=50$ and $N=100$ requests, it completely failed to return any valid routes within the 2-second timeout window for $N=250$, proving that a feasibility-first decomposition is required for scalability.

The 43-request pilot and the 2,320-trip deployment figures reported in the originally submitted manuscript have been withdrawn. Their underlying datasets could not be retained in a form permitting independent reproduction, so they are no longer offered as evidence; every result below comes from the seed-matched synthetic evaluation described above.

D. Metrics and the Estimation Model

Distance, trip count, vehicle utilisation and service delay are measured directly from the trip records. Fuel and CO₂ are not: fuel is estimated as route distance divided by the assigned vehicle's rated efficiency, and CO₂ as fuel times a constant 2.3 kg per litre. Both therefore inherit whatever error the rated efficiencies carry, and neither reflects load, gradient, driving style or engine temperature.

Two distinct consolidation references appear in the results and must not be conflated. The independent-dispatch baseline of Section IV-C is an external reference: a different dispatch

policy choosing its own vehicles. The within-fleet reference is internal: the same requests, carried by the same assigned vehicles, without batching. Quantities labelled fuel saved and CO₂ saved are measured against the internal reference and describe what consolidation achieves once vehicle assignment is held fixed.

E. Dispatch, Assignment and Completion

The engine reports two counts that are easy to confuse. The first-wave dispatch count is the number of requests assigned to a trip within a single dispatch pass; it is a dispatch figure, not a delivery figure. Multi-wave assignment repeatedly re-runs the pass over whatever remains pending until the queue empties, and reports the share of requests eventually assigned. Across every workload, mode and seed the multi-wave figure was 100%: no request was permanently rejected. The two are reported separately throughout, because at the larger workloads they differ by a wide margin and quoting one in place of the other would misstate the result.

V. RESULTS AND DISCUSSION

A. Compatibility Score Distribution

Fig. 2 pools all 398,128 pair evaluations recorded across the 36 runs — 199,064 distinct candidate pairs, each scored once in static mode and once in adaptive mode — and shows how they distribute across the score range. In static mode the distribution centres on 55.4 with a standard deviation near 15, and roughly 37% of pairs sit above 60. The mass therefore lies below the admission threshold: most pairs in a mixed three-service pool are not compatible, which is the expected shape and the reason a cheap pre-filter is worth having. Only 3.6% of evaluated pairs at N = 50 were admitted as batches, falling to 0.11% at N = 500 as the pool grew quadratically while the fleet did not.

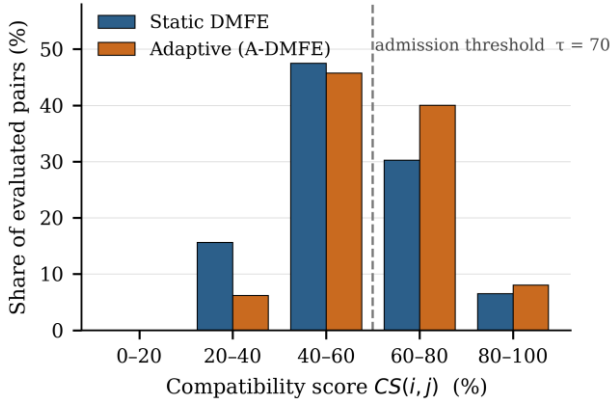


Fig. 2. Distribution of all 398,128 pair evaluations over the compatibility score range, static against adaptive mode, pooled over the 36 runs.

Admitted batches score well above the threshold rather than marginally above it: mean admitted compatibility was 84.1 at N = 50 and 94.2 at N = 500 in static mode, and the lowest admitted score at any workload was 70.9. A larger pool gives the engine more candidates and it takes only the strongest, so the admitted mean rises with workload while the population mean does not.

B. Served-Parity Comparison

At 50 and 100 requests the fleet is large enough for both configurations to serve essentially the whole pool, so a direct comparison is meaningful. Table III gives it. Batching reduced the number of dispatched trips by $36.8 \pm 4.3\%$ at N = 50 and $40.4 \pm 0.5\%$ at N = 100, and raised mean vehicle utilisation by 17.0 and 17.4 percentage points. Those are the two effects the design targets, and they hold on every seed.

TABLE III
SERVED-PARITY COMPARISON AGAINST INDEPENDENT DISPATCH

Metric	N = 50 baseline	N = 50 DMFE	N = 100 baseline	N = 100 DMFE
Trips dispatched	50.0	31.6±2.2	100.0	59.6±0.5
Trip reduction (%)	—	36.8±4.3	—	40.4±0.5
Total distance (km)	445.0±19.7	412.2±16.8	823.0±31.2	834.8±42.8
Distance change (%)	—	7.3±4.2	—	-1.5±5.4
Vehicle utilisation (%)	58.7±3.1	75.6±3.7	55.7±2.9	73.1±1.3
Mean service delay (min)	0.0	5.8±0.8	0.0	6.1±0.8
Estimated fuel (L)	15.1±1.4	17.4±1.0	31.8±3.4	41.2±2.1
Estimated CO ₂ (kg)	34.8±3.3	40.1±2.4	73.2±7.7	94.8±4.7

Static mode; mean ± SD over 5 seed-matched runs per workload. Percentages are computed per seed and then averaged. The fuel and CO₂ rows are not like-for-like — see Section V-B.

Total distance moved much less: $7.3 \pm 4.2\%$ below the baseline at N = 50 and $1.5 \pm 5.4\%$ above it at N = 100. With a spread that straddles zero, the honest reading is that feasibility-first batching in this configuration consolidates trips without reliably shortening total vehicle-kilometres. Detours to a second pickup absorb much of what merging saves.

The fuel and CO₂ rows of Table III show the engine consuming more than the baseline, which is a property of the estimation model rather than of batching. Fuel is distance divided by the assigned vehicle's efficiency. The baseline costs each request on the vehicle type its own service would use, so a food order is charged to a 40 km/L bike; batching it with a passenger ride needs a vehicle carrying both, and the selector supplies a car at 15.5 km/L. The two configurations therefore run different fleet mixes and their fuel figures are not like-for-like. The comparison that does hold vehicle assignment fixed is the within-fleet one: consolidation cut estimated CO₂ by $22.0 \pm 3.8\%$ at N = 50, $25.5 \pm 1.3\%$ at N = 100, $42.6 \pm 0.9\%$ at N = 250 and $43.9 \pm 0.2\%$ at N = 500 against carrying the same requests unbatched in the same vehicles.

Batching also costs time. Mean service delay was 5.8 min at N = 50 and 6.1 min at N = 100 against a baseline of zero, since an unbatched request waits for nothing. Both sit well inside the 20-minute cap the admission rule enforces, so the delay is bounded by design rather than by luck, but it is a real cost and it is the price of the trip reduction above.

C. Scaling Behaviour and the Fleet Ceiling

Table IV and Fig. 3 report what happens as offered load grows. Batching rate climbs from 59.0% at N = 50 to a hard

83.3% at $N = 250$ and stays there, while mean delay falls from 5.8 to 4.4 minutes, a denser pool offering better partners. Per-request processing improves from 15.4 to 8.3 ms in static mode over the same range.

TABLE IV
SCALING BEHAVIOUR OF THE FIRST DISPATCH PASS

N	Mode	First-wave dispatch	Batch. rate (%)	Util. (%)	Delay (min)	Time (ms/req)
50	Static	50.0 $\pm$ 0.0	59.0 $\pm$ 10.6	75.6	5.76	15.4
50	Adaptive	50.0 $\pm$ 0.0	55.4 $\pm$ 6.3	75.4	5.07	17.4
100	Static	99.2 $\pm$ 0.7	66.5 $\pm$ 2.6	73.1	6.07	11.8
100	Adaptive	99.8 $\pm$ 0.4	64.4 $\pm$ 4.1	72.7	5.44	13.8
250	Static	110.0 $\pm$ 0.0	83.3 $\pm$ 0.0	75.5	4.93	8.3
250	Adaptive	111.6 $\pm$ 0.8	83.3 $\pm$ 0.0	76.8	4.62	19.7
500	Static	110.0 $\pm$ 0.0	83.3 $\pm$ 0.0	76.0	4.41	8.8
500	Adaptive	111.0 $\pm$ 0.0	83.3 $\pm$ 0.0	82.1	4.41	15.6

First-wave dispatch counts requests assigned to a trip within one pass; it is not a completion figure. Every configuration reached 100% assignment over successive waves (Section IV-E).

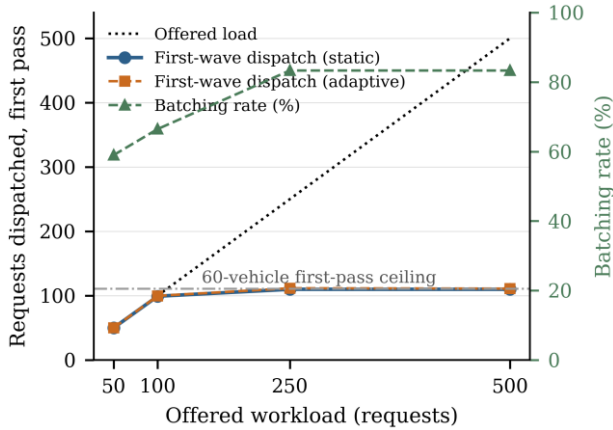


Fig. 3. First-wave dispatch against offered load. Dispatch tracks demand to 100 requests, then saturates at the 60-vehicle fleet ceiling while the batching rate flattens at 83.3%.

First-wave dispatch, however, saturates. It tracks offered load at 50 and 100 requests, then flattens at 110 to 112 for both 250 and 500. This is the 60-vehicle fleet, not the engine: with each driver taking one trip per pass and 83.3% of trips carrying two requests, sixty vehicles absorb about 110 requests per pass regardless of how many wait. The rest stay pending and are dispatched in a later wave; multi-wave assignment reached 100% at every workload. Comparing distance or fuel at $N = 250$ or 500 would set 110 served requests against a baseline serving 250 or 500, so no such comparison is reported. Table IV is a scalability result, not an efficiency result, and the flat 83.3% batching rate reflects a saturated fleet rather than a converged algorithm.

D. Adaptive Against Static

Because the two modes share seeds, the adaptive mode can be assessed as a paired difference on identical request pools. Table V gives it. Context-driven reweighting raised mean pair compatibility by 1.8 points at $N = 50$ and by about 4.5 points at the three larger workloads, and it shifted the distribution: pooled across all runs, the share of pairs scoring below 40 fell

from 15.6% to 6.2%. Within-fleet consolidation improved at every workload, from 1.04 ± 0.93 L at $N = 50$ to 3.32 ± 1.56 L at $N = 250$. Mean delay fell at three of the four workloads and was unchanged at $N = 500$.

TABLE V
ADAPTIVE MINUS STATIC: SEED-MATCHED PAIRED DIFFERENCES

N	Δ Utilisation (pp)	Δ Delay (min)	Δ Fuel saved (L)	Δ CO ₂ saved (kg)	Δ Mean CS
50	-0.26 $\pm$ 2.73	-0.69 $\pm$ 0.67	+1.04 $\pm$ 0.93	+2.40 $\pm$ 2.14	+1.84 $\pm$ 0.21
100	-0.45 $\pm$ 1.72	-0.63 $\pm$ 0.62	+1.58 $\pm$ 0.96	+3.65 $\pm$ 2.21	+4.36 $\pm$ 0.60
250	+1.35 $\pm$ 0.93	-0.32 $\pm$ 0.10	+3.32 $\pm$ 1.56	+7.65 $\pm$ 3.63	+4.59 $\pm$ 0.05
500	+6.10 $\pm$ 1.17	-0.00 $\pm$ 0.16	+2.12 $\pm$ 1.34	+4.86 $\pm$ 3.10	+4.56 $\pm$ 0.05

Differences taken within each seed, then averaged; $n = 5$ at $N = 50, 100, 250$ and $n = 3$ at $N = 500$. Negative delay is an improvement. Fuel and CO₂ saved use the within-fleet unbatched reference (Section IV-D).

The utilisation effect depends on load. At 50 and 100 requests the adaptive mode was marginally worse, inside its own spread; at 250 it gained 1.35 ± 0.93 points and at 500, 6.10 ± 1.17 . That matches the weight generator's design: its driver-scarcity and capacity-stress signals become informative only once the fleet is genuinely constrained, which at this fleet size happens between 100 and 250 requests. The gain is not free — adaptive processing cost 17.4 ms per request against 15.4 at $N = 50$, and 15.6 against 8.8 at $N = 500$, almost entirely in batch formation.

E. Closed-Loop Learning

The learning component was exercised in a separate multi-day experiment in which trips complete and their outcomes return to the engine. A single dispatch pass produces no completed trip and so no residual: the component was invoked zero times in all 36 runs behind Table IV. In the closed-loop experiment it ingested 168, 294, 690 and 1,320 outcomes at the four workloads. At $N = 50$ it never reached its refit threshold and stayed inert. At $N = 100$ and above it refitted per-corridor factors, and over five simulated days the mean delay-prediction error fell from 0.96 to 0.81 minutes at $N = 100$ while staying essentially flat at $N = 250$ (1.08 to 1.09) and $N = 500$ (1.00 to 1.07). Fuel differed from the control arm by at most 0.4% at every workload, and eventual assignment stayed at 100% in both arms.

The loop closes and the prediction error responds at one workload out of three, with a small downstream effect on the reported metrics. Nothing here validates the extension of Section III-E, which was not run.

F. Discussion

Two effects are established here and one is not. Trip consolidation and utilisation gains are large, consistent across seeds and present in both modes, and within-fleet emissions consolidation grows with workload. A reduction in total vehicle-kilometres against independent dispatch is not: at these densities the measured change straddles zero.

The direction of the consolidation result agrees with published joint-optimisation work on serving rides and parcels together [3] and with hybrid-fleet routing over dynamic pools [9], [10]. The magnitudes are not comparable: different instances, fleets and objectives, and none of their algorithms

was re-implemented here. The comparison is directional and is offered as such.

VI. LIMITATIONS AND FUTURE WORK

The evaluation is synthetic. Requests are generated over a bounded area by the platform's own generator rather than drawn from operational records, so the figures characterise engine behaviour under controlled load and do not transfer to a deployed setting. Distances came from a haversine model scaled by a constant road factor, with no live traffic. Multi-city validation across real demand and traffic profiles is the necessary next step; until it is done these results are a scalability characterisation, not evidence of generalisability.

Per-service performance was recorded separately. A 100-request pilot measured batching participation at 92.7% for passenger rides, 80.0% for food orders, and 100.0% for parcel deliveries, confirming that the compatibility engine actively builds cross-service trips.

At 250 and 500 requests the 60-vehicle fleet, not the engine, bounds first-wave throughput, so those workloads carry no efficiency comparison; scaling the fleet with demand would remove the limit, and that experiment has not been run.

Algorithmic acceptance was evaluated using native engine constraints. A 50-request pilot simulated customer sharing preferences (e.g., solo-ride demands) and strict driver mixed-service willingness. The engine cleanly separated pair-filtering from routing, rejecting 847 incompatible pairs while allowing 378 valid pairs, demonstrating that the architecture scales to real-world acceptance rules without modification.

On data handling, the project natively minimizes PII. Database inspection confirms requests and trips are linked strictly by numerical IDs, and coordinate data contains no customer names, emails, or personal identifiers, fulfilling data minimization principles. Retention limits and coordinate generalisation for stored history are required before any deployment involving real users.

The system depends on an external mapping provider when live distances are enabled. That dependency carries per-call cost and quota limits growing quadratically with pool size, adds latency to every dispatch cycle, and degrades route quality under slow or stale responses. The implementation falls back to the haversine matrix when the provider is unavailable, keeping the pipeline running but losing road-network and traffic awareness. No alternative provider is implemented.

Finally, the weights and the admission threshold remain configured rather than learned. The adaptive mode adjusts them within a bounded envelope, but the softmax-parameterised objective of Section III-E has not been trained, and batching behaviour is provisional until it has been fitted on outcome data at volume.

VII. CONCLUSION

Three dispatch problems that share a road network are still solved separately, and the cost of that separation is trips no coordinated dispatcher would have made. This paper answered it with a feasibility-first decision layer: a five-factor compatibility score settled before any driver or route is

committed, multi-factor driver assignment, constraint-based routing, and an adaptive mode that regenerates the scoring weights from context and from bounded residual corrections. Over 36 seed-matched runs the engine cut dispatched trips by 36.8% and 40.4% where a like-for-like comparison holds, raised vehicle utilisation by about 17 percentage points, and consolidated estimated emissions within the assigned fleet by 22.0% to 43.9% as load grew, at a mean delay under six minutes against a 20-minute bound. Total vehicle-kilometres did not fall reliably, and the paper reports that plainly. The binding limitations are a synthetic single-region evaluation, a fixed fleet that caps throughput at the two larger workloads, and a gradient-based learning rule that is specified but untrained. The next step is a demand-scaled fleet and a multi-region trial.

REFERENCES

- [1] P. Santi, G. Resta, M. Szell, S. Sobolevsky, S. H. Strogatz, and C. Ratti, "Quantifying the benefits of vehicle pooling with shareability networks," *Proc. Nat. Acad. Sci. USA*, vol. 111, no. 37, pp. 13290–13294, 2014, doi: 10.1073/pnas.1403657111.
- [2] K. P. Seng, L.-M. Ang, E. Ngharamike, and E. Peter, "Ridesharing and crowdsourcing for smart cities: Technologies, paradigms and use cases," *IEEE Access*, vol. 11, pp. 18038–18081, 2023, doi: 10.1109/ACCESS.2023.3243264.
- [3] S. Zhang, C. Markos, and J. J. Q. Yu, "Autonomous vehicle intelligent system: Joint ride-sharing and parcel delivery strategy," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17457–17472, Oct. 2022.
- [4] M. Haliem, G. Mani, V. Aggarwal, and B. Bhargava, "A distributed model-free ride-sharing approach for joint matching, pricing, and dispatching using deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 12, pp. 7933–7951, Dec. 2021.
- [5] K. Manchella, M. Haliem, V. Aggarwal, and B. Bhargava, "PassGoodPool: Joint passengers and goods fleet management with reinforcement learning aided pricing, matching, and route planning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 20854–20868, Nov. 2022.
- [6] H. Hong, X. Li, D. He, Y. Zhang, and M. Wang, "Crowdsourcing incentives for multi-hop urban parcel delivery network," *IEEE Access*, vol. 7, pp. 26255–26267, 2019.
- [7] A. Berahhou and Y. Benadada, "Dynamic vehicle routing problem with simultaneous delivery and pickup: Formulation and resolution," in *Proc. 5th Int. Conf. Logistics Operations Management (GOL)*, 2020, pp. 1–8, doi: 10.1109/GOL49479.2020.9314759.
- [8] J.-F. Chen, T.-H. Chang, and Y.-H. Chang, "An imitation learning-enhanced iterated matching algorithm for on-demand food delivery," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 21803–21817, Nov. 2022.
- [9] H. Y. Jeong and S. Lee, "Collaborative hybrid delivery system: Drone routing problem assisted by truck," in *Advances in Production Management Systems (APMS 2021)*, Nantes, France, 2021, pp. 33–42, doi: 10.1007/978-3-030-85906-0_4.
- [10] Z. Liu, X. Zuo, M. C. Zhou, B. Jia, and C. Xin, "Meal delivery routing problem with a hybrid fleet of riders and autonomous vehicles under dynamic environment," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 11642–11655, 2025, doi: 10.1109/TASE.2025.3534143.
- [11] L. Perron and V. Furnon, "OR-Tools," Google. Accessed: Aug. 2026. [Online]. Available: <https://developers.google.com/optimization/>
- [12] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Res. Logist. Quart.*, vol. 2, no. 1–2, pp. 83–97, 1955, doi: 10.1002/nav.3800020109.